

Java 登入視窗程式碼除錯與修正

<https://gemini.google.com/app/d24990e8993a9de4>

User prompt: 產生「一個有輸入框和按鈕的登錄視窗」得到的結果，這段程式碼有一些錯誤，幫我糾錯，要明確說出來哪裡錯誤，之後幫我改成正確且可以執行的程式碼

Response: 圖片中的這段 Java Swing 程式碼確實存在幾個重大問題，導致視窗無法正常顯示內容或邏輯判定失效。以下為您詳細說明錯誤原因與改善建議：

程式碼中的主要錯誤分析

1. 元件座標未設定 (setLayout(null) 但無 setBounds)

- 問題：程式中寫了 `setLayout(null);` (絕對定位)，但加入視窗的 `t1`, `t2`, `btn` 都沒有呼叫 `setBounds(x, y, width, height)` 設定位置與大小。
- 結果：元件預設的大小會是 0×0 ，導致畫面上什麼都看不到。
- 建議：改用靈活的版面配置管理器 (如 `GridLayout` 或 `FlowLayout`)，或者為每個元件手動設定 `setBounds`。

2. 字串比較使用了 == 運算子

- 問題：在 Java 中，字串 (String) 是物件，`==` 比較的是記憶體位址而非字串的**內容**。
- 結果：即使輸入了 "admin"，`t1.getText() == "admin"` 依然會回傳 `false`，導致永遠無法登入成功。
- 建議：應使用 `.equals()` 方法進行字串內容比較，例如 `t1.getText().equals("admin")`。

3. setVisible(true) 呼叫時機過早

- 問題：在建構子一開始時就呼叫了 `setVisible(true)`；當時畫面的元件都還沒新增 (add) 進去。
- 結果：視窗繪製時可能無法及時更新剛剛加入的元件，造成畫面空白或顯示異常。
- 建議：將 `setVisible(true)` 移至所有元件都已建立並 add 到視窗後的最後一步。

4. 密碼輸入框型態不恰當與邏輯問題

- 問題：密碼欄位 `t2` 使用了普通的 `JTextField`，輸入時密碼會直接以明文顯示；且條件式中的 `t2.getText() == ""` 不完整或無遮罩。
- 建議：密碼欄位應改用 `JPasswordField` 進行安全遮罩。

5. 缺少視窗關閉處置 (setDefaultCloseOperation)

- 問題：沒有設定 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`。
- 結果：點擊視窗右上角關閉視窗後，Java 背景程序依然會繼續執行，不會完全結束。

修正後的完整可執程式碼

這裡為您提供兩種修正方式：

方式一：使用 LayoutManager (標準推薦做法)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;

public class GoodLogin extends JFrame {
    private JTextField t1;
    private JPasswordField t2;

    public GoodLogin() {
```

```

setTitle("登入");
setSize(300, 180);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null); // 視窗置中顯示

// 使用網格版面配置 (3列2行)
setLayout(new GridLayout(3, 2, 10, 10));

JLabel l1 = new JLabel("帳號:", SwingConstants.CENTER);
t1 = new JTextField();
JLabel l2 = new JLabel("密碼:", SwingConstants.CENTER);
t2 = new JPasswordField(); // 使用密碼輸入框

JButton btn = new JButton("登入");

// 加入元件
add(l1);
add(t1);
add(l2);
add(t2);
add(new JLabel("")); // 預留空位排版
add(btn);

// 監聽器與字串比較修正
btn.addActionListener((ActionEvent e) -> {
    String username = t1.getText();
    String password = new String(t2.getPassword());

    // 正確的字串內容比較方式 .equals()
    if ("admin".equals(username) && "123456".equals(password)) {
        JOptionPane.showMessageDialog(this, "登入成功!");
        System.out.println("登入成功");
    } else {
        JOptionPane.showMessageDialog(this, "帳號或密碼錯誤", "錯誤", JOptionPane.ERROR_MESSAGE);
    }
});

// 所有元件與監聽器均設定完畢後，最後才顯示視窗
setVisible(true);
}

public static void main(String[] args) {
    // Swing 官方建議的線程安全寫法
    SwingUtilities.invokeLater(() -> new GoodLogin());
}
}

```

方式二：保留 null 排版（手動指定絕對座標 setBounds）

如果您希望沿用原本的絕對排版（setLayout(null)），則必須手動為每一個元件設定位置與尺寸：

```

import javax.swing.*;

public class GoodLoginAbsolute extends JFrame {
    public GoodLoginAbsolute() {
        setTitle("登入");
        setSize(320, 220);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null); // 使用絕對定位

        JLabel l1 = new JLabel("帳號:");
        l1.setBounds(30, 20, 50, 25);

        JTextField t1 = new JTextField();
        t1.setBounds(90, 20, 160, 25);
    }
}

```

```

JLabel l2 = new JLabel("密碼:");
l2.setBounds(30, 60, 50, 25);

JPasswordField t2 = new JPasswordField();
t2.setBounds(90, 60, 160, 25);

JButton btn = new JButton("登入");
btn.setBounds(90, 110, 100, 30);

add(l1);
add(t1);
add(l2);
add(t2);
add(btn);

btn.addActionListener(e -> {
    String user = t1.getText();
    String pass = new String(t2.getPassword());

    if ("admin".equals(user) && "123456".equals(pass)) {
        JOptionPane.showMessageDialog(this, "登入成功!");
    } else {
        JOptionPane.showMessageDialog(this, "帳號或密碼錯誤!");
    }
});

// 放在最後顯示
setVisible(true);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new GoodLoginAbsolute());
}
}

```

User prompt: 我的檔案名稱改成hw1

Response: 如果要把類別與檔案名稱改為 hw1，在 Java 中 **類別名稱 (class name)** 必須與檔名一致。因此，類別名稱需要從 BadLogin 改為 hw1，檔名則存為 hw1.java。

以下為修改後的完整可執行程式碼：

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;

public class hw1 extends JFrame {
    private JTextField t1;
    private JPasswordField t2;

    public hw1() {
        setTitle("登入");
        setSize(300, 180);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 點擊關閉按鈕時完全結束程式
        setLocationRelativeTo(null); // 視窗顯示於螢幕中央

        // 使用 3x2 的網格版面配置
        setLayout(new GridLayout(3, 2, 10, 10));

        JLabel l1 = new JLabel("帳號:", SwingConstants.CENTER);
        t1 = new JTextField();
        JLabel l2 = new JLabel("密碼:", SwingConstants.CENTER);
        t2 = new JPasswordField(); // 密碼欄位使用遮罩
    }
}

```

```
        JButton btn = new JButton("登入");

        // 將元件加入視窗中
        add(l1);
        add(t1);
        add(l2);
        add(t2);
        add(new JLabel("")); // 佔位空組件以維持排版
        add(btn);

        // 按鈕事件處理
        btn.addActionListener((ActionEvent e) -> {
            String username = t1.getText();
            String password = new String(t2.getPassword());

            // 使用 .equals() 正確比較字串內容
            if ("admin".equals(username) && "123456".equals(password)) {
                JOptionPane.showMessageDialog(this, "登入成功!");
                System.out.println("登入成功");
            } else {
                JOptionPane.showMessageDialog(this, "帳號或密碼錯誤", "錯誤", JOptionPane.ERROR_MESSAGE);
            }
        });

        // 放在最後確保所有元件均已加入後再顯示視窗
        setVisible(true);
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new hw1());
    }
}
```

注意事項

- **檔案命名**：請將此程式碼儲存為 hw1.java（注意大小寫）。
- **編譯與執行指令**：

```
javac hw1.java
java hw1
```

User prompt: 那除了這些問題外 還有需要修改的地方嗎

Response: 除了前面提到的版面配置、字串比較與 setVisible 時機等核心問題外，從**代碼品質（Code Quality）**、**Swing 最佳實踐與安全性的角度**來看，還有以下幾個值得修改與優化的細節：

1. 變數命名規範（Naming Conventions）

- **問題**：原程式碼中的變數名如 l1, t1, l2, t2, btn 太過簡略且缺乏語意。
- **修改建議**：採用具備描述性的駝峰式命名（CamelCase），例如將 t1 改為 usernameField，t2 改為 passwordField，btn 改為 loginButton。這能大幅提升程式碼的可讀性與維護性。

2. 類別名稱首字母大寫（Java 命名規範）

- **問題**：Java 的類別名稱（Class Name）慣例上應採用 **PascalCase**（首字母大寫）。雖然 hw1 能正常編譯與執行，但寫成小寫不符合 Java 的標準規範。
- **修改建議**：建議將類別名改為 Hw1，檔案命名為 Hw1.java。

3. 硬編碼（Hardcoded）字串應提取為常數或設定檔

- **問題：**帳號與密碼的比對值（如 "admin"）直接寫死在事件處理器中。
- **修改建議：**至少將預設的帳號密碼提取為 `private static final` 常數，方便未來維護或擴充：

```
private static final String CORRECT_USERNAME = "admin";
private static final String CORRECT_PASSWORD = "123";
```

4. 忘記去除輸入文字的前後空白（Trimming Input）

- **問題：**使用者在輸入帳號時，偶爾會不小心複製到空格或多打空格，導致驗證失敗。
- **修改建議：**取得帳號文字時呼叫 `.trim()` 處理：

```
String username = usernameField.getText().trim();
```

5. 密碼記憶體安全問題（使用 `char[]` 處理密碼）

- **問題：**`JPasswordField` 的 `getText()` 方法已被官方宣告廢棄（Deprecated），因為 `String` 在 Java 中是不可變物件（Immutable），會停留在記憶體堆疊中直到被垃圾回收，有洩漏風險。
- **修改建議：**改用 `passwordField.getPassword()` 取得 `char[]`，比對完畢後手動清空陣列內容：

```
char[] password = passwordField.getPassword();
// 驗證完畢後清空
java.util.Arrays.fill(password, '0');
```

6. Swing 執行緒安全性（Thread Safety）

- **問題：**原程式碼在 `main` 方法中直接執行 `new BadLogin()`。Java Swing 元件的建立與更新應該要在 **Event Dispatch Thread (EDT)** 中進行。
- **修改建議：**在 `main` 方法中使用 `SwingUtilities.invokeLater()` 啟動 GUI：

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new Hw1());
}
```

綜合優化後的完整程式碼（Hw1.java）

```
import javax.swing.*;
import java.awt.*;
import java.util.Arrays;

public class Hw1 extends JFrame {

    // 1. 預設驗證資料（常數）
    private static final String ADMIN_USER = "admin";
    private static final String ADMIN_PASS = "123456";

    // 2. 使用具備描述性的變數名稱
    private final JTextField usernameField;
    private final JPasswordField passwordField;

    public Hw1() {
        // 基本視窗設定
        setTitle("登入系統");
        setSize(320, 180);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // 設定關閉動作
        setLocationRelativeTo(null); // 置中顯示

        // 版面配置
        setLayout(new GridLayout(3, 2, 10, 10));

        // 初始化元件
        JLabel usernameLabel = new JLabel("帳號:", SwingConstants.CENTER);
```

```
usernameField = new JTextField();

JLabel passwordLabel = new JLabel("密碼:", SwingConstants.CENTER);
passwordField = new JPasswordField();

JButton loginButton = new JButton("登入");

// 將元件加入容器
add(usernameLabel);
add(usernameField);
add(passwordLabel);
add(passwordField);
add(new JLabel("")); // 排版用空元件
add(loginButton);

// 事件處理
loginButton.addActionListener(e -> performLogin());

// 3. 所有元件設置完畢後才顯示視窗
setVisible(true);
}

private void performLogin() {
    String inputUser = usernameField.getText().trim();
    char[] inputPass = passwordField.getPassword();

    // 安全地比較密碼 (將常數轉為 char[] 進行比較)
    boolean isUserCorrect = ADMIN_USER.equals(inputUser);
    boolean isPassCorrect = Arrays.equals(ADMIN_PASS.toCharArray(), inputPass);

    if (isUserCorrect && isPassCorrect) {
        JOptionPane.showMessageDialog(this, "登入成功!", "訊息", JOptionPane.INFORMATION_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "帳號或密碼錯誤!", "錯誤", JOptionPane.ERROR_MESSAGE);
    }

    // 清理記憶體中的密碼敏感資料
    Arrays.fill(inputPass, '0');
}

public static void main(String[] args) {
    // 4. 使用 EDT 線程安全的呼叫方式
    SwingUtilities.invokeLater(Hw1::new);
}
}
```
